

Binary Tree Representation & Types -- Data Structures

Definition: A Binary Tree is a hierarchical tree data structure in which every node has at most two children, referred to as the left child and the right child.

1. Detailed Technical Explanation

1. Types of Binary Trees

1. FULL (PROPER) BINARY TREE

```
      [ 1 ]
     /   \
    [ 2 ] [ 3 ]
   /  \  /  \
  [ 4 ] [ 5 ]
```

(Every node has 0 or 2 children)

2. COMPLETE BINARY TREE

```
      [ 1 ]
     /   \
    [ 2 ] [ 3 ]
   /  \  /
  [ 4 ] [ 5 ] [ 6 ]
```

(All levels full except last, filled L-to-R)

3. PERFECT BINARY TREE

```
      [ 1 ]
     /   \
    [ 2 ] [ 3 ]
   /  \  /  \
  [4] [5][6] [7]
```

(All internal nodes have 2 children, all leaves at same level)

4. DEGENERATE / SKEWED TREE

```
      [ 1 ]
       \
        [ 2 ]
         \
          [ 3 ]
```

(All nodes have only 1 child; behaves like a linked list)

1. Full (Proper/Strict) Binary Tree: Every node has either 0 or 2 children. No node has only 1 child.

2. Co
for Bin

2. Binary Tree Representations in Memory

1. Sequential (Array-Based) Representation

For a complete binary tree stored in array A (1-indexed):

* No

2. Linked (Pointer-Based) Representation in C

```
#include <stdio.h>
#include <stdlib.h>

// Structure for a Binary Tree Node
struct TreeNode {
    int data;
    struct TreeNode* left;
    struct TreeNode* right;
};

// Function to create a new tree node
```

```
struct TreeNode* createNode(int value) {
    struct TreeNode* newNode = (struct TreeNode*)malloc(sizeof(struct TreeNode));
    newNode->data = value;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}
```

3. Mathematical Properties of Binary Trees

- * Maximum nodes at level i (root at level 0) = 2^i .

* Ma

4. Must-Write Points for Exams

- * In a Full Binary Tree, no node has degree 1.

* In

5. Quick Recall Flow

Binary Tree (≤ 2 Children) -> Types: Full ($0/2$), Complete (L-to-R), Perfect, Skewed -> Array ($2i, 2i+1$) vs Linked Node